

Big Data Analysis & Project

Assignment 1 D

Sai Akshay Suresh

a1906525

Crime Clearance Status Prediction with Deep Learning & Gradient Boosting Models

Abstract

The work investigates the efficiencies of the deep learning models, namely, TabTransformer, TabNet, and the machine learning models, based on gradient boosting methods such as Light Gradient Boosting Machine (LightGBM), Cat Boost, and Histogram Gradient Boosting Classifier (HistGradientBoosting) for predicting the crime clearance statuses (Cleared “C”, Not Cleared “N”), of the criminal cases registered in the Austin Police Department, Texas in the first four months (January-April) of 2025 based on the data from (2003-2024). This work explores the performance of the models, in which the dataset has an imbalance, where the instances when the cases are cleared “C” are fewer than the cases that are not cleared “N”, and the “N” is considered the majority class. This work also examines the capabilities of the models to work with cyclic features (like month of a year, hour of a day, etc) that are sine- and cosine-transformed, and the performances are evaluated with the help of necessary performance indicators such as accuracies, losses, sensitivities, specificities, F1 score, precision, etc. This study highlights the capabilities of machine learning models over deep learning models in terms of performance and reduced computational cost.

1. Introduction

In a country, in order to ensure that the law enforcement agencies, especially the police, are performing their duty, the crime clearance rates are important. High crime clearance rates can help the communities to believe that the police system is effective and vigilant in delivering justice. This builds trust, fosters cooperation, and reinforces the relationship between police and citizens. Conversely, low clearance rates pave the way to community distrust and disinterest of the public to cooperate to the law (Solving Crime, Protecting Communities: A Blueprint for Safer Streets - R Street Institute 2025). Clearing crimes often sends a message that offenders are likely to be held responsible, which prevents potential criminals. While low clearance rates could weaken this crime prevention strategy, and can increase criminal behaviour (The Impact of Clearance Rates on Communities, Number Analytics 2025). Studies reveal that clearance rates are quite lower for certain demographic groups-particularly homicides involving victims of African descent, and this reflects broader issues of bias and inequality in the justice system (Clayton, 2024). Lower crime clearance rates also indicate complex system-based challenges, like a shortage of staff, obsolete investigative tools, or overburdening the personnel. These issues can potentially erode public confidence in police, along with the reduction of effectiveness (Pressley & Jackson, 2024).

In Chicago, based on a community study, it was found that both actual and perceived homicide clearance rates contribute significantly to the increase in likelihood that residents will cooperate with the police, like dialling the number after witnessing a homicide or even providing suspect information. Simply, if people notice that the cases are being solved, they are more willing to help the law agencies (Bocanegra & Gottlieb, 2025). The transparent reporting of the crime-solving rates by the agencies can potentially reduce underreporting of crimes. Prediction of crime clearance rates can empower the agencies to upscale their investigative efforts strategically, by marking cases likely to remain unsolved or spotlighting the discrepancies earlier, thereby increasing the overall efficiency and effectiveness (Kar 2024). Forecasting clearance rates highlights surging trends-such as declines due to forensic lag, staffing, or community disengagement, and provides an immediate basis for policy intervention. Studies indicate that such insights can help the institutions aim at reversing downward trends in clearance rates (Predictive Policing Forecasting Crime for Law Enforcement n.d.).

To predict the crime clearance rate of an institution, the Deep Learning (DL) and Machine Learning (ML) models have shown significantly promising capacities to handle complex, high-dimensional crime data and uncover patterns. A recent review of ML and DL techniques shows how both these practices have become foundational in crime prediction modelling, using historical crime, spatial, temporal, and socio-economic data (Hinduja et al. 2025). On one hand, DL models can identify nuances and non-linear relationships in crime data, particularly when geographic and time-based features exist. For example, crime forecasting models with LSTM and CNN architectures have demonstrated stronger pattern finding abilities and higher predictive accuracies than conventional ML models like Random Forest, Logistic Regression, etc (Predictive Crime Analytics Using Deep Learning Approaches for Crime Forecasting and Prevention T.ANTO TEEPAAK, CHANDRU M, Associate Professor of IT 2 nd M TECH n.d.). On the other hand, ML models, specifically, Gradient Boosting Models, offer high predictive abilities and lower computational demands, especially useful in imbalanced data. These Gradient boosting algorithms are basically ensemble methods, i.e, based on multiple predictions, the most reliable outcome is picked, and this helps in the reduction of errors. These models are widely favorable in explainable crime prediction contexts-such as in the prediction of homicide clearance, and ensuring a useful performance (Campedelli 2022). To extract more authentic results, transformation of cyclic variables like (months, hours, etc.) using sine and cosine encoding, for preservation of their periodic nature and enhanced model learning, particularly in DL and ML networks. Such encoding is intended to make the model recognize the continuity (eg, January-December) rather than treating them as mutually exclusive categories. Moreover, in a supervised learning setting, the models benefit from enhanced convergence and stability when cyclic variables are transformed properly (Lewinson 2022).

This work addresses the crime prediction task by the following accomplishments-

- The Deep Learning Techniques, designed specifically for dealing with tabular data with numerical (including the transformed variables) and categorical variables (with high cardinality), are identified (TabNet and TabTransformer) and implemented.
- The Machine Learning Models, designed to handle datasets with class imbalances, and also with an ability to process numerical and categorical data, are found (LightGBM, CatBoost, and HistGradientBoostingClassifier), and implemented.
- These 5 models had been evaluated based on the performance metrics, namely, Training, Testing, Validation Accuracies, Losses, Sensitivity, Specificity, F1-Score, and

Precision. Randomly sampled prediction results were taken for sanity checking of the implemented models.

2. Literature Review

A comparison table of the relevant works, focusing on the implemented models and equivalent works, in terms of the performance indicators, is presented in Table 1 below. The implemented methods for this study are highlighted. It is to be noted that the highest performance scores of the test set are only used for comparison.

Work	Model (Type)	Task	Accuracy	Sensitivity	Specificity	Precision	F1-Score
(Yingze et al. 2024)	TabNet (DL)	Landslide Susceptibility Forecasting	79.8%	0.898	-	0.187	0.309
(Pillai n.d.)		Cardiac Disease Prediction	94.4%	More than 0.93	More than 0.93	0.93	-
Proposed Work		Crime Clearance Prediction	74.03%	0.939	0.153	0.460	0.23
(Setu et al. 2025)	TabTransformer (DL)	Cyberattack Detection	-	0.85	-	0.89	0.86
(Application of Tabular Transformer Architectures for Operating System Fingerprinting 2017)		OS fingerprinting from network flow features (multi-level classification)	75.59%	0.756	-	0.817	0.759
Proposed Work		Crime Clearance Prediction	72.90%	0.881	0.279	0.443	0.342
(Mireia Ladios-Martin et al. 2025)	LightGBM (ML)	Calculating the Risk of Admission to Intensive Care Units in COVID-19 Patients Using Machine Learning	98%	0.92	0.99	-	0.93
(Liu et al. 2023)		Differentiate AL amyloidosis from other conditions using EHR/lab features.	82%	0.92	0.60	-	0.73
Proposed Work		Crime Clearance Prediction	89.25%	0.948	0.728	0.912	0.929
(Islam, Vega-Corredor & Wilson 2025)	HistGradientBoosting (ML)	Binary classification of wetland vs non-wetland from satellite imagery	89%	0.65	-	0.80	0.72
(Moon, Ahsan & Anwar 2024)		Classify tweets/news as “shared” vs “not shared” (Fake news detection).	91.07%	0.944	-	0.876	0.909

Proposed Work		Crime Clearance Prediction	89.17%	0.956	0.702	0.905	0.929
(Yar et al. 2025)	CatBoost (ML)	Classify COVID-19 presence and identify risk factors.	97.79%	0.972	0.986	0.978	0.987
(Peykani et al. 2025)		Predict business failure (credit risk) for firms in the Iranian capital market.	-	0.909	-	0.201	0.329
Proposed Work		Crime Clearance Prediction	88.47%	0.943	0.712	0.906	0.924

Table 1, Literature Review Summary of the Proposed Work with a Comparison of the Existing Works

3. Research Methodology

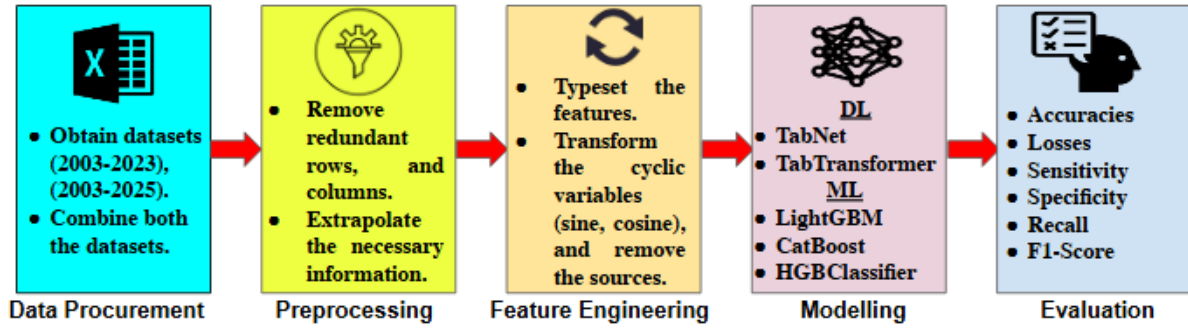


Figure 1, Flow diagram of the Study

3.1 Data Procurement, Preprocessing, and Feature Engineering

As shown above in Figure 1, the research was conducted in 5 phases. The dataset was collected from two different sources, namely Gigasheet, which had crime report information of 2.36 million rows and 36 columns, along with their resolution status from the year 2003 to 2023, and the Government of Austin, Texas's Database, which had crime report information of 2.61 million rows and 19 columns, along with their resolution status from the year 2003 to 2025. These two datasets were combined, and a few columns were extrapolated from the time-based features, such as the date, time crime occurred, and date, time crime reported. A few unnecessary columns in the post crime clearance scenario, such as crime clearance date, Uniform Crime Reporting (UCR) category, category description were removed as a part of preprocessing. The extrapolated columns from the occurrence date, time, and report date, time were cyclic (bound to get repeated) like day of the month, month in a year, hour of a day, minute of an hour, week of the year, day of the week, report time (in numbers like 2159, 010, etc), and occurred time (in numbers). These were encoded with both sine, cosine transformations with the following formulae-

$$\text{sine}_{\text{cyclic variable}}(x, \text{total_val}) = \frac{\sin(2\pi)}{\text{total_val}} \quad (1)$$

$$\text{cosine}_{\text{cyclic variable}}(x, \text{total_val}) = \frac{\cosine(2\pi)}{\text{total_val}} \quad (2)$$

The total_val is the total number of possible values in a cyclic variable. For eg, the number of months in a year is 12, the number of minutes in an hour is 60, the number of days in a week is 7, etc. After the conversion of the cyclic variables with sine and cosine encodings, the source variables from which they were converted are removed. These sine and cosine encoded features were numerical representations. There were 30 numerical and 7 categorical columns in the resultant datasets, as the representations from the year 2025 were separately taken as the test dataset. The train dataset (2003 – 2024) had 117338 rows, and the test dataset (2025) had 10427 rows, as the original dataset with 4.9 million rows was sampled with the student number (1906525), which is also used for splitting the train dataset with 80% for training, and 20% for validation. The features were set to the appropriate types and are represented in Table 2, below.

Sl. Number	Column Name	Format	Description	Notes
1	highest_offense_description	Factor	351 types of offenses.	-
2	highest_offense_code	Factor	316 types of offense codes, where for a few offenses of the same nature, we can find the same code that groups them.	-
3	family_violence	Factor	2 types – Y or N family violence tags.	-
4	occurred_date_time_year	Numeric	Years from 2003 to 2025.	-
5	sine_odt_month	Numeric.	The sine transformed value of the incident occurred month. ODT means occurred date time.	Extracted from occurred_date_time_month, which had months of the year (Jan-Dec). And this original column was removed after encoding.
6	cosine_odt_month	Numeric.	The cosine transformed value of the incident occurred month.	Extracted from occurred_date_time_month, which had months of the year (Jan-Dec). And this original column was removed after encoding.
7	sine_odt_week_of_year	Numeric.	The sine transformed value of the incident occurred week of the year.	Extracted from occurred_date_time_week_of_year, which had 1-53 weeks in a year. And this original column was removed after encoding.
8	cosine_odt_week_of_year	Numeric.	The cosine transformed value of the incident occurred week of the year.	Extracted from occurred_date_time_week_of_year, which had 1-53 weeks in a year. And this original column was removed after encoding.
9	sine_odt_day	Numeric.	The sine transformed value of the incident occurred day of the month.	Extracted from occurred_date_time_day, which had 1-31 days in a month. And this original column was removed after encoding.
10	cosine_odt_day	Numeric	The cosine transformed value of the incident occurred day of the month.	Extracted from occurred_date_time_day, which had 1-31 days in a month. And this original column was removed after encoding.
11	sine_odt_day_of_week	Numeric	The sine transformed value of the incident occurred day of the week.	Extracted from occurred_date_time_day_of_week, which had 7 days of the week, Monday-Sunday. And this original column was removed after encoding.
12	cosine_odt_day_of_week	Numeric	The cosine transformed value of the incident occurred day of the week.	Extracted from occurred_date_time_day_of_week, which had 7 days of the week, Monday-Sunday. And this original column was removed after encoding.

13	sine_odt_hour	Numeric	The sine transformed value of the incident occurred hour in a day.	Extracted from occurred_date_time_hour, which had (0-23) of the 24 hours in a day. And this original column was removed after encoding.
14	cosine_odt_hour	Numeric	The cosine transformed value of the incident occurred hour in a day.	Extracted from occurred_date_time_hour, which had (0-23) of the 24 hours in a day. And this original column was removed after encoding.
15	sine_odt_minute	Numeric	The sine transformed value of the incident occurred minute in an hour.	Extracted from occurred_date_time_minute, which had (0-59) in 60 minutes of an hour. And this original column was removed after encoding.
16	cosine_odt_minute	Numeric	The cosine transformed value of the incident occurred minute of an hour.	Extracted from occurred_date_time_minute, which had (0-59) in 60 minutes of an hour. And this original column was removed after encoding.
17	sine_occurred_time	Numeric	The sine transformed value of the incident occurred time (as plain numbers).	Extracted from occurred_time, which had the times of incidents as plain numbers (00-2359). And this original column was removed after encoding.
18	cosine_occurred_time	Numeric	The cosine transformed value of the incident occurred time (as plain numbers).	Extracted from occurred_time, which had the times of incidents as plain numbers (00-2359). And this original column was removed after encoding.
19	sine_report_time	Numeric	The sine transformed value of the incident reported time (as plain numbers).	Extracted from report_time, which had the times of incidents as plain numbers (00-2359). And this original column was removed after encoding.
20	cosine_report_time	Numeric	The cosine transformed value of the incident reported time (as plain numbers).	Extracted from report_time, which had the times of incidents as plain numbers (00-2359). And this original column was removed after encoding.
21	report_date_time_year	Numeric	The year in which the incident was reported (2003-2025).	-
22	sine_rdt_week_of_year	Numeric	The sine transformed value of the incident reported week of the year.	Extracted from report_date_time_week_of_year, which had 1-53 weeks in a year. And this original column was removed after encoding.
23	cosine_rdt_week_of_year	Numeric	The cosine transformed value of the incident reported week of the year.	Extracted from report_date_time_week_of_year, which had 1-53 weeks in a year. And this original column was removed after encoding.
24	sine_rdt_day	Numeric	The sine transformed value of the incident reported day of the month.	Extracted from report_date_time_day, which had 1-31 days in a month. And this original column was removed after encoding.
25	cosine_rdt_day	Numeric	The cosine transformed value of the incident reported day of the month.	Extracted from report_date_time_day, which had 1-31 days in a month. And this original column was removed after encoding.
26	sine_rdt_hour	Numeric	The sine transformed value of the incident reported hour in a day.	Extracted from report_date_time_hour, which had (0-23) of the 24 hours in a day. And this original column was removed after encoding.
27	cosine_rdt_hour	Numeric	The cosine transformed value of	Extracted from report_date_time_hour, which had (0-23) of the 24 hours in a

			the incident reported hour in a day.	day. And this original column was removed after encoding.
28	sine_rdt_minute	Numeric	The sine transformed value of the incident reported minute in an hour.	Extracted from report_date_time_minute, which had (0-59) in 60 minutes of an hour. And this original column was removed after encoding.
29	cosine_rdt_minute	Numeric	The cosine transformed value of the incident reported minute in an hour.	Extracted from report_date_time_minute, which had (0-59) in 60 minutes of an hour. And this original column was removed after encoding.
30	sine_rdt_month	Numeric	The sine transformed value of the incident reported month in a year.	Extracted from report_date_time_month, which had months of the year (Jan-Dec). And this original column was removed after encoding.
31	cosine_rdt_month	Numeric	The cosine transformed value of the incident reported month in a year.	Extracted from report_date_time_month, which had months of the year (Jan-Dec). And this original column was removed after encoding.
32	sine_rdt_day_of_week	Numeric	The sine transformed value of the incident reported day of the week.	Extracted from report_date_time_day_of_week, which had 7 days of the week, Monday-Sunday. And this original column was removed after encoding.
33	cosine_rdt_day_of_week	Numeric	The cosine transformed value of the incident reported day of the week.	Extracted from report_date_time_day_of_week, which had 7 days of the week, Monday-Sunday. And this original column was removed after encoding.
34	location_type	Factor	The 40 location types of the crime incidents.	-
35	apd_sector	Factor	The 9 types of Austin Police Department sectors: AD, BA, CH, DA, ED, FR, GE, HE, ID.	-
36	apd_district	Factor	The 8 types of Austin Police Department districts (1-8).	-
37	clearance_status (Response Variable)	Factor	The two types of clearance statuses, Cleared "C", and Not Cleared "N".	-

Table 2, The Feature Summary of the Dataset.

As shown below, in Table 3, the train and test datasets were imbalanced, with “N” being greater.

Train Data

Clearance Status	Count	Proportion
C	42454	36.18%
N	74884	63.82%
Total	117338	100.00%

Test Data

Clearance Status	Count	Proportion
C	2637	25.29%
N	7789	74.71%
Total	10426	100.00%

Table 3, The Dataset Summary of Number and Proportions of Labels in Train (Left), and Test (Right) Datasets.

3.2 Modelling and Evaluation

After an extensive feature engineering, the two DL models, TabNet and TabTransformer, and the three ML models, CatBoost, HistGradientBoosting (HGB) Classifier, and LightGBM, were evaluated for ablation studies with two common hyperparameters that are valid across all the models, despite subtle differences in their names. The hyperparameters chosen were iteration-specific to learn the model’s features well and model-specific to analyse the model’s functioning in various settings, as given below in Table 4. It is to be noted that each of the models was run for one model-based hyperparameter; all the iteration-based hyperparameters are tested. For eg, for a na=nd value of 8, the epochs 5, 10, 20, 35, 50 are tested, followed by the same fashion for all the na=nd values.

Model Name	Hyperparameter Name	Model-Specific Iteration-Specific	or	Values
TabNet	na=nd	Model-Specific		8, 16, 32, 48, 64
	Number of Epochs	Iteration-Specific		5,10,20,35,50
TabTransformer	Number of Embeddings	Model-Specific		8,16,32,64,128
	Epochs	Iteration-Specific		5,10,20,35,50
LightGBM	Max Depth	Model-Specific		3, 5, 7, 10, -1
	Number of Estimators	Iteration-Specific		50, 100, 200, 400, 800
CatBoost	Max Depth	Model-Specific		3, 5, 7, 10, 12
	Number of Iterations	Iteration-Specific		50, 100, 200, 400, 800
HGB Classifier	Max Depth	Model-Specific		3, 5, 7, 10, None
	Number of Iterations	Iteration-Specific		50, 100, 200, 400, 800

Table 4. Hyperparameter Summary for Models

The DL model, specifically TabNet, was selected due to its ability to use sequential attention to focus on the most relevant features at each decision step, thus enabling both interpretability and efficient learning from tabular data, and supports self-supervised pre-training to enhance performance when unlabelled data is available (Arik & Pfister 2020). The hyperparameters na (attention embedding dimension), nd (decision embedding dimension) have been set equal with the same values, as this is a standard procedure used to capture richer feature interactions (Kądziołka et al. 2024). The increment of epochs could make sure that the features are learned, trained, and validated thoroughly before testing. The TabTransformer, chosen due to its transformer-based contextual embeddings for categorical features with high cardinality and robustness to any missing or noisy data, helps capture more intricate details than simple one-hot encodings (Huang et al. 2020). The larger embeddings can help the model in identifying nuances in the categorical data, but are accompanied by the risk of overfitting (Yury Gorishniy et al. 2021). The epochs, for TabTransformer, help improve convergence, despite having the same problem of overfitting as embeddings.

The Gradient Boosting Decision Tree (GBDT) ML models, especially the LightGBM model, has been studied for its high speed computation capacity and the scalability feature, which promotes leaf-based growth and histogram-based grouping to handle large datasets efficiently (mljourney 2024). LightGBM is highly effective in time-series-based forecasting tasks and outperforms other GBDT models when the hyperparameters are tuned (Barros, Cerqueira & Soares 2021). The increase in depth, shown to be beneficial for understanding complex feature relationships, and the “-1”, indicating no depth, has reduced error according to the study (Barros, Cerqueira & Soares, 2021). Despite the increase in training time, the existing work (Baron Yugovich 2024) shows that the increment of the number of estimators can improve the predictive accuracy.

The CatBoost GDBT model was picked to study, as it introduced ordered boosting and native categorical handling, unlike LightGBM, which was introduced very recently. These features help in the reduction of target leaks and improve generalization (Chinmay Shripad Kulkarni, 2022). CatBoost has been promising for datasets with a higher number of categorical features (CatBoost: unbiased boosting with categorical features - statwiki 2015). The hyperparameter depth, with an increase, can produce a model's improved expressiveness (a neural network's capacity to model a wide range of complex tasks), and it was explored from parameter tuning guides that a depth between 1-10 can manage complexity, with the reduction of overfitting (Filho 2023). The iterations (trees) can engage the model in better learning, and a study found that it had a significant impact on the model's performance in terms of R-squared (Yaren Aydın, Gebrail Bekdaş & Sinan Melih Nigdeli 2024).

The HistGradientBoosting (HGB) classifier uses a histogram-based gradient boosting strategy to accelerate training and reduce the cost of computation for large-scale data (GeeksforGeeks 2024). When the depth is fixed to "None", an constrain-free tree grows, and offers flexibility in understanding the features at the cost of a potential overfitting (SKLearner 2024). The research (scikit-learn 2025) shows, to restrict the depth values in the 3-10 range to bypass overfitting, and potentially underfitting as well. The iterations here are the boosting rounds, which increase the learning capacity of the model. A standard starting point is ~100, and a higher number of iterations can result in underperformance (Restackio 2025).

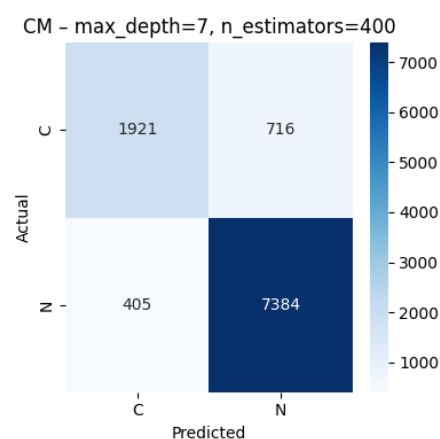


Figure 2, Confusion Matrices for LightGBM Model (400 Estimators and depth of 7), True Positive (Bottom-Right), False Positives (Top-Right), False Negatives (Bottom-Left), True Negatives (Top-Left).

The confusion matrices, as shown in Figure 2, have played an important role in the extraction of performance indicators. It is to be observed that the models considered, the majority class, which is not cleared “N” as positive, and cleared “C” as negative.

The discussed models have been evaluated with various key performance metrics, as provided below in Table 5-

Sl. No	Metric	Formulae/Methods used
1.	Training, Testing, Validation Accuracies.	These accuracies are the number of right predictions of the total number of predictions on the train, validation, and test sets.
2.	Training, Testing, Validation Losses.	These log likelihood losses measure how closely the predicted outcomes comply with the actual outcomes. It is to be interpreted that the higher values of losses are less accurate, and the lower values indicate that the predictions are nearer to the actual outcomes.

3.	Sensitivity/Recall/True Positive Rate	Proportion of true positives that are correctly identified by the model of the total number of correct predictions. $\text{Sensitivity} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$
4.	Specificity	Proportion of true negatives that are correctly identified by the model of the total number of correct predictions. $\text{Specificity} = \frac{\text{True Negatives}}{\text{True Negatives} + \text{False Positives}}$
5.	Precision	Proportion of the true positives on the total number of positive predictions. $\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$
6.	F1 Score	A harmonic mean of the sensitivity and precision. $\text{Precision} = \frac{2 * (\text{Precision} * \text{Sensitivity})}{\text{Precision} + \text{Sensitivity}}$

Table 5, Key Performance Indicators used for Evaluation of Models

4. Experimental Evaluation

4.1 Experimental Setup

The data processing works, including the data inspection, preprocessing, and feature engineering, were done in R with a personal computer of the following processing capabilities-

- **Processor:** Intel(R) Core(TM) i5-1035G1 CPU @ 1.00GHz (1.19 GHz)
- **Installed RAM:** 8.00 GB (7.79 GB usable)
- **System type:** 64-bit operating system, x64-based processor

The modelling part has been done on Google Colab (Python) with an L4 GPU of the following features (TechPowerUp, 2024)-

- **Architecture:** NVIDIA Ada Lovelace architecture, built on the 5nm process.
- **Memory:** 24 GB of GDDR6 memory with a 192-bit memory interface.
- **Core Configuration:**
 - 7424 shading units (CUDA cores)
 - 240 texture mapping units
 - 80 ROPs (Render Output Units)
 - 240 Tensor Cores for accelerating machine learning tasks
 - 60 Ray Tracing (RT) Cores for enhanced graphics rendering
- **Clock Speeds:**
 - Base Clock: 795 MHz
 - Boost Clock: Up to 2040 MHz
 - Memory Clock: 1563 MHz (12.5 Gbps effective)
- **Power Consumption:** Maximum power draw of 72W.
- **Form Factor:** Single-slot card, designed for high-density server environments.

- **Driver and CUDA Compatibility:** Requires NVIDIA driver version 525.0 or later and CUDA toolkit version 12.0 or later.

4.2 Experimental Evaluation

na_nd	Epoch	Tr_acc	Te_acc	Tr_loss	Te_loss	Prec	F1	Spec	Sens
8	5	82.73%	74.04%	0.467	0.813	0.460	0.231	0.154	0.939
32	5	84.75%	72.96%	0.384	0.726	0.280	0.076	0.044	0.962
48	50	89.09%	70.88%	0.282	0.881	0.397	0.336	0.291	0.850
32	50	88.79%	70.53%	0.289	0.799	0.325	0.208	0.153	0.892
48	5	83.93%	69.77%	0.401	0.623	0.285	0.177	0.129	0.890

Table 6, Top-5 TabNet Performances through Hyperparameter variations, arranged by Testing Accuracies

The TabNet model has struggled with recognizing the minority “C” class, as evidenced by its poor specificities and F1 scores in Table 6, above. With the increase of attention and dimension-based embeddings, there is an increase in the training accuracy from 82% to above 85% approximately. The model suffers from overfitting, as the training data is learned well, but struggles to have an impact on the testing data, as seen above.

Embedding dimensions	Epoch	Tr_acc	Te_acc	Tr_loss	Te_loss	Prec	F1	Spec	Sens
64	5	87.58%	72.90%	0.313	0.651	0.443	0.342	0.279	0.881
32	5	87.44%	71.88%	0.316	0.617	0.364	0.213	0.150	0.911
8	20	88.02%	69.46%	0.302	0.653	0.380	0.353	0.329	0.818
64	20	89.70%	69.24%	0.258	inf	0.333	0.261	0.215	0.854
16	5	87.45%	68.40%	0.316	0.726	0.323	0.267	0.228	0.838

Table 7, Top-5 TabTransformer Performances through Hyperparameter variations, arranged by Testing Accuracies

The TabTransformer model, as shown above in Table 7, also suffers from a similar case to TabNet. Despite a slight increase in specificities, F1, and precisions, the model provides a fatal validation, and test loss of infinity in fewer cases when the embedding dimensions are increased along with epochs. This shows the limitation of the TabTransformer model for an imbalanced dataset, even with the variations in dimensions and epochs, suggesting enormous overfitting. The TabNet leads TabTransformer in terms of accuracy, managing losses despite the enhancements of TabTransformer in minority class detection. The deep learning models have shown subpar performance for crime clearance status detection.

Depth	Estimators	Tr_acc	Te_acc	Tr_loss	Te_loss	Prec	F1	Spec	Sens
7	400	92.48%	89.25%	0.198	0.274	0.912	0.929	0.728	0.948
5	100	89.82%	89.19%	0.261	0.272	0.906	0.930	0.708	0.954
-1	200	90.94%	88.95%	0.229	0.272	0.907	0.928	0.714	0.949
7	800	94.82%	88.00%	0.149	0.281	0.909	0.927	0.721	0.946
10	400	92.52%	88.00%	0.195	0.276	0.909	0.927	0.719	0.946

Table 8, Top-5 LightGBM Performances through Hyperparameter variations, arranged by Testing Accuracies

The LightGBM, as represented in Table 8 above, has shown a promising performance in terms of all the aspects from the earlier Deep Learning models. The rapid increase in training accuracy, along with testing accuracy, shows that the model has powerful prospects when identifying the “C” class. As discussed in the earlier section, with the “-1” value for depth, the model has strived to achieve an outstanding training accuracy of over 95% along with a testing accuracy of around 88%. Also, with the

increase of depth from 3 to 10, the model was getting better in terms of scarce class prediction. As researched earlier, LightGBM has surpassed the deep learning models in all aspects, also with an added advantage of computational feasibility, taking a lower training time.

Depth	Iterations	Tr_acc	Te_acc	Tr_loss	Te_loss	Prec	F1	Spec	Sens
7	50	89.32%	88.47%	0.274	0.285	0.906	0.924	0.712	0.943
5	100	89.09%	88.43%	0.279	0.285	0.906	0.924	0.710	0.943
10	800	91.03%	88.17%	0.237	0.289	0.907	0.922	0.717	0.937
12	800	92.16%	87.51%	0.211	0.296	0.904	0.918	0.708	0.932
3	800	89.05%	87.49%	0.279	0.291	0.907	0.917	0.720	0.927

Table 9, Top-5 CatBoost Performances through Hyperparameter variations, arranged by Testing Accuracies

The CatBoost model, as illustrated by Table 9 above, has shown an identical but marginally lower performance to LightGBM. With the increment of depth from 3 to 10, the model has improved training accuracy from 88 to over 90%. The precisions, F1, and specificities, as seen in LightGBM, have had a good improvement from the DL models in recognizing the minority cleared status class. Despite LightGBM taking the lead over CatBoost, the lower gap between training and testing accuracies shows that the model's learning capabilities from the training set are properly applied in the testing set, making CatBoost the model for more reliable predictions.

Depth	Iterations	Tr_acc	Te_acc	Tr_loss	Te_loss	Prec	F1	Spec	Sens
5	100	89.86%	89.17%	0.260	0.275	0.905	0.930	0.702	0.956
None	100	89.98%	89.05%	0.255	0.277	0.904	0.929	0.700	0.955
7	50	89.51%	89.04%	0.273	0.281	0.902	0.929	0.692	0.958
5	50	89.31%	89.03%	0.276	0.283	0.901	0.929	0.688	0.959
3	200	89.37%	87.34%	0.272	0.288	0.900	0.917	0.692	0.935

Table 10, Top-5 HGB Classifier Performances through Hyperparameter variations, arranged by Testing Accuracies

The Histogram-based Gradient Boosting Classifier has also shown stronger results in terms of sensitivity and F1 scores, by surpassing both CatBoost and LightGBM marginally as seen in Table 10. Despite these improvements and the variations of depth and iterations, the model has had repeatedness in results, as only at a depth of 5, the model performance slightly differs with variations in iterations. This could mean that the model is robust, and the hyperparameter change does not matter. The setting of depth to "None" meant that the results were not as good with the depth of values above 5, and the HGB Classifier also lacks generalizability in this setting.

5. Discussion

Model	Tr_acc	Te_acc	Prec	F1	Spec	Sens
TabNet	82.73%	74.04%	0.460	0.231	0.154	0.939
TabTransformer	87.58%	72.90%	0.443	0.342	0.279	0.881
LightGBM	92.48%	89.25%	0.912	0.929	0.728	0.948
CatBoost	89.32%	88.47%	0.906	0.924	0.712	0.943
HGB Classifier	89.86%	88.91%	0.905	0.930	0.702	0.956

Table 11, Best Performances of Deep Learning and Machine Learning Models for Crime Clearance Detection

As seen before, it is evident that LightGBM emerges as the ultimate winner in terms of accuracy, precision, and specificity, which is reiterated again in Table 11. But, Histogram-based Gradient Boosting Classifier results in a marginally superior result than LightGBM when it comes to F1-score and sensitivity, doing a great job in the majority "N" class detection. The average accuracies throughout all the hyperparameters for the tested models, as given below in

Figure 3, emphasize that the Gradient Boosting ML Models, especially LightGBM, HGB Classifier, and CatBoost, have had a superior working tendency than the Deep Learning Models, and, it was observed from an external study that the neural networks struggle to learn irregular patterns of the target variable, and the high cardinality categorical variables also hurt their performance, particularly when dealing with numerous less important features in tabular data (Grinsztajn, Oyallon & Varoquaux 2022). This has resulted in poor testing accuracy for TabNet, and TabTransformer.

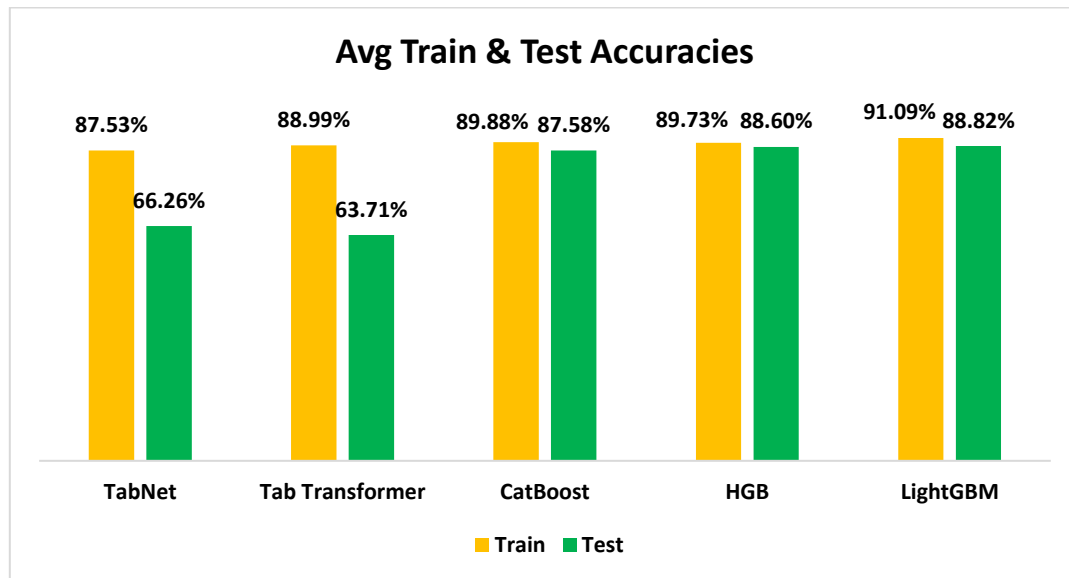


Figure 3, Average Train & Test Accuracies for all the Models throughout Hyperparameter Sweeps

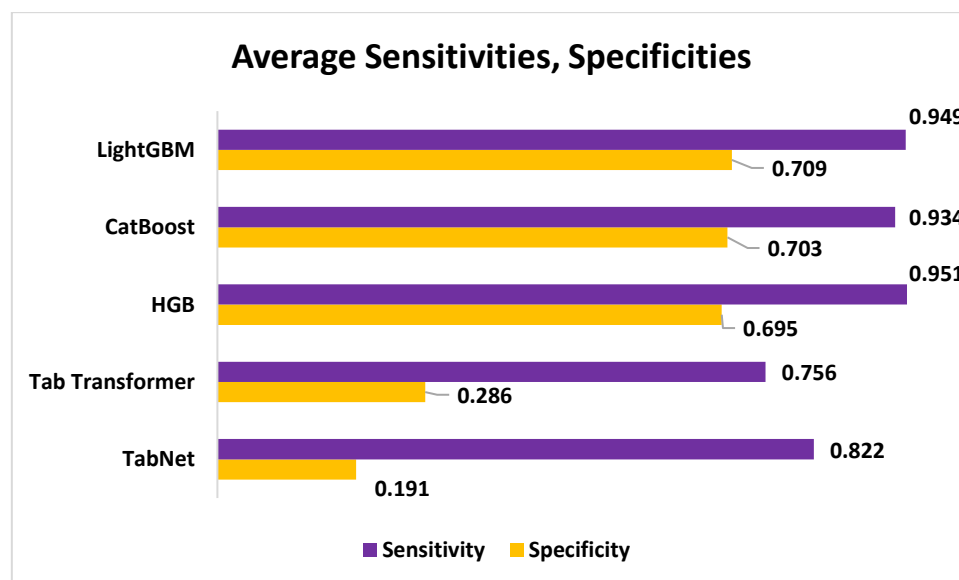


Figure 4, Average Sensitivities & Specificities for all the Models throughout Hyperparameter Sweeps

As seen above, from Figure 4, despite having decent sensitivities, the DL models struggle with the sensitivity of the minority “C” class detections. This could be a reason for DL mode’s biases toward the majority class “N”, and high detection of False Positives. This situation is quite common in neural network training, especially with imbalanced data, where the models optimize the aggregate losses and give over-importance to the majority classes, thus resulting

in a misaligned learned decision boundary against the minority class samples (Ghosh et al. 2022), (Chen et al. 2024). The GDBT ML models has done a decent job in minority class detection with overall specificities ~ 0.70 surpassing the DL models.

6. Limitations

The conducted research has had a few shortcomings, as given below

- The HGB Classifier has a limit to deal with only 250 categories. As two of the vital features `highest_offense_description`, and `highest_offense_code` consisted of more than 300 categories, the categories had to be grouped into 250 categories. This is because HGB uses histogram-based discretization for high-cardinality features (Brownlee 2020).
- The struggles of DL models and ML models' limited ability to identify the minority "C" classes indicate that, because of the default optimization strategies and maximization of node purity, prioritization of recall of the minority classes is missed. Without finetuning with cost-sensitive learning or resampling, the specificity can not be increased and tends to plateau (GeeksforGeeks 2024a).
- The CatBoost model gave an out-of-memory error when the tree depth was set to 16. It was studied that the CatBoost's default depth "n" created 2^n leaves per tree, so the depth 16 can branch out 65,536 leaves per tree, and this makes all per-leaf histograms explode by exhausting the RAM. It is recommended to set with much shallower depths than 16, like 6 to 10, on small datasets, in order to avoid potential memory blow-up (FAQ 2018).

7. Conclusion

To conclude, the Gradient Boosting Decision Tree-based ML models, notably LightGBM, and Histogram Gradient Boosting Classifier, demonstrated their abilities to work with an imbalanced dataset, outshining the DL models like TabTransformer, and TabNet for the prediction of crime clearance status in the first four months of 2025 based on the insights gathered from (2003-2024) in Austin Police Department, Texas. All the tested models performed exceptionally well in finding the majority class "N". To scale the performance, one can try using the custom class weighting modes or the induction of focal loss, along with the application of optimizers as hyperparameters for the DL models, and improve the minority class detection in GDBT models. Furthermore, for fine-tuning the performance, leveraging the hybrid models, which encompass the abilities of DL and GDBT-ML models, can be used to understand the advantages and pitfalls of working in an imbalanced dataset. For a dependable clearance status prediction based on past data in a supervised learning setting, the GDBT-ML models, LightGBM, HGB classifier, followed by CatBoost are recommended.

8. Codes & Repository

The codes, linked to the drive repository, can be found here at the privately available link: https://github.com/a1906525/Big-Data_a1906525.git

9. References

Arik, SO & Pfister, T 2020, 'TabNet: Attentive Interpretable Tabular Learning', *arXiv:1908.07442 [cs, stat]*.

Application of Tabular Transformer Architectures for Operating System Fingerprinting 2017, *arXiv.org*, viewed 11 August 2025, <https://arxiv.org/html/2502.09084v1>.

Baron Yugovich 2024, 'How many trees do I actually have in my LightGBM model?', *Stack Overflow*.

Barros, FS, Cerqueira, V & Soares, C 2021, 'Empirical Study on the Impact of Different Sets of Parameters of Gradient Boosting Algorithms for Time-Series Forecasting with LightGBM', *Lecture Notes in Computer Science*, pp. 454–465.

Bocanegra, K & Gottlieb, A 2025, 'A Community-Based Examination of the Association between Perceived and Actual Homicide Clearance Rates and Cooperation with Homicide Investigations', *American Journal of Criminal Justice*, Springer Science+Business Media.

Brownlee, J 2020, *Histogram-Based Gradient Boosting Ensembles in Python*, Machine Learning Mastery.

CatBoost: unbiased boosting with categorical features – statwiki 2015, *Uwaterloo.ca*, viewed 12 August 2025, https://wiki.math.uwaterloo.ca/statwiki/index.php?title=CatBoost%3A_unbiased_boosting_with_categorical_features.

Chen, W, Yang, K, Yu, Z, Shi, Y & Chen, P 2024, 'A survey on imbalanced learning: latest research, applications and future directions', *Artificial Intelligence Review*, vol. 57, Springer Science+Business Media, no. 6.

Clayton, A 2024, 'Two Black moms say police won't solve their children's murders. A new study asks: is race to blame?', *The Guardian*, online, 21 February, available at: <https://www.theguardian.com/us-news/2024/feb/21/black-homicide-clearance-rate-lower-than-white>.

FAQ 2018, *FAQ* | *Catboost.ai*, viewed 13 August 2025, <https://catboost.ai/docs/en/concepts/faq?>.

Filho, M 2023, 'CatBoost Hyperparameter Tuning Guide with Optuna', *Forecastegy.com*, Forecastegy, viewed 12 August 2025, <https://forecastegy.com/posts/catboost-hyperparameter-tuning-guide-with-optuna?>.

GeeksforGeeks 2024, 'HistGradientBoostingClassifier in Sklearn', *GeeksforGeeks*, viewed 12 August 2025, <https://www.geeksforgeeks.org/machine-learning/histgradientboostingclassifier-in-sklearn/>.

GeeksforGeeks 2024a, 'Training a decision tree against unbalanced data', *GeeksforGeeks*, viewed 13 August 2025,

<https://www.geeksforgeeks.org/machine-learning/training-a-decision-tree-against-unbalanced-data/>.

Ghosh, K, Bellinger, C, Corizzo, R, Branco, P, Krawczyk, B & Japkowicz, N 2022, 'The class imbalance problem in deep learning', *Machine Learning*, vol. 113.

Grinsztajn, L, Oyallon, E & Varoquaux, G 2022, 'Why do tree-based models still outperform deep learning on tabular data?', *arXiv.org*, viewed 13 August 2025, <https://arxiv.org/abs/2207.08815?>.

Hinduja, P, Tejasree, M, Harini, M & Babu, R 2025, 'Crime Prediction Using Machine Learning', *International Journal of Research Publication and Reviews*, vol. 6, no. 2, Journal homepage: www.ijrpr.com.

Huang, X, Khetan, A, Cvitkovic, M & Karnin, Z 2020, *TabTransformer: Tabular Data Modeling Using Contextual Embeddings*, *arXiv.org*, viewed 12 August 2025, <https://arxiv.org/abs/2012.06678?>.

Islam, S, Vega-Corredor, MC & Wilson, MD 2025, 'Mapping Wetlands with High-Resolution Planet SuperDove Satellite Imagery: An Assessment of Machine Learning Models Across the Diverse Waterscapes of New Zealand', *Remote Sensing*, vol. 17, Multidisciplinary Digital Publishing Institute, no. 15, pp. 2626–2626, viewed 11 August 2025, <https://www.mdpi.com/2072-4292/17/15/2626>.

Islam, S (no date), *Cardiac Disease Prediction with Tabular Neural Network*, viewed 11 August 2025, <https://www.ijert.org/research/cardiac-disease-prediction-with-tabular-neural-network-IJERTV11IS110099.pdf>.

Kądziołka, B, Jurkiewicz, P, Wójcik, R & Domżał, J 2024, 'Utilizing TabNet Deep Learning for Elephant Flow Detection by Analyzing Information in First Packet Headers', *Entropy*, vol. 26, no. 7, p. 537.

Kar, S 2024, *Predictive Policing – Using Data to Target Crime, Not Communities*, IGNESA, viewed 11 August 2025, <https://ignesa.com/predictive-policing/>.

Lee, S (no date), 'The impact of clearance rates on communities', *Number Analytics // Super Easy Data analysis tool for Research*, available at: <https://www.numberanalytics.com/blog/impact-of-clearance-rates-on-communities> (Accessed: 11 August 2025).

Lewinson, E 2022, 'Three Approaches to Encoding Time Information as Features for ML Models', *NVIDIA Technical Blog*.

Liu, Y, Dou, X, Wen, L, Gao, X, Wang, X & Lu, J 2023, 'Applying Machine Learning to Support Early Diagnosis of Light-Chain Amyloidosis: A Combination of Knowledge-Based Approach with Data-Driven Approach', *Blood*, vol. 142, no. Supplement 1, pp. 2038–2038.

Mireia Ladios-Martin, María José Cabañero-Martínez, José Fernández-de-Maya, Francisco-Javier Ballesta-López, Garcia-Garcia, I, Adrián Belso-Garzas, Francisco-Manuel Aznar-Zamora & Cabrero-García, J 2025, 'Calculating the Risk of Admission to Intensive Care Units in COVID-19 Patients Using Machine Learning', *Journal of Clinical Medicine*, vol. 14, Multidisciplinary Digital Publishing Institute, no. 12, pp. 4205–4205, viewed 11 August 2025, <https://www.mdpi.com/2077-0383/14/12/4205>.

mljourney 2024, *LightGBM Classifier: Features, Implementation, and Best Practices – ML Journey*, ML Journey.

Moon, UF, Ahsan, M & Anwar, MM 2024, 'Modeling the Sharing and Diffusion of Fake News in Social Media', viewed 11 August 2025, <https://ieeexplore.ieee.org/abstract/document/10940155>.

Peykani, P, Peymany Foroushany, M, Tanasescu, C, Sargolzaei, M & Kamyabfar, H 2025, 'Evaluation of Cost-Sensitive Learning Models in Forecasting Business Failure of Capital Market Firms', *Mathematics*, vol. 13, MDPI AG, no. 3, p. 368.

Predictive Crime Analytics Using Deep Learning Approaches for Crime Forecasting and Prevention, T.ANTO TEEPAK, CHANDRU M Associate Professor of IT 2nd M TECH (no date), viewed 11 August 2025, https://ijirt.org/publishedpaper/IJIRT168800_PAPER.pdf.

Predictive Policing Forecasting Crime for Law Enforcement (no date).

Pressley, N & Jackson, R 2024, *Ensuring Justice: Innovative Approaches to Improve Crime Clearance Rates*, Right on Crime.

Pillai, S (no date), *Cardiac Disease Prediction with Tabular Neural Network*, viewed 11 August 2025, <https://www.ijert.org/research/cardiac-disease-prediction-with-tabular-neural-network-IJE-RTV11IS110099.pdf>.

Setu, JH, Alam, A, Halder, N, Mahmood, A, Islam, A & Amin, MA 2025, 'IoT Cyberattack Detection via TabTransformer on CAN Bus Communications', *Lecture Notes in Electrical Engineering*, Springer Nature Singapore, Singapore, pp. 59–65.

Solving Crime, Protecting Communities: A Blueprint for Safer Streets – R Street Institute 2025, R Street Institute, viewed 11 August 2025, <https://www.rstreet.org/research/solving-crime-protecting-communities-a-blueprint-for-safer-streets>.

TechPowerUp 2024, *NVIDIA L4 Specs*, online, available at: <https://www.techpowerup.com/gpu-specs/l4.c4091> (Accessed 12 August 2025).

Yar, T, Ipek Cicek, Gul, T & Karaman, U 2025, 'Determination of risk factors for COVID-19 patients using the CatBoost machine learning technique', *Medicine Science | International Medical Journal*, vol. 14, Society of Turaz Bilim, no. 1, pp. 134–134, viewed 11 August 2025, <https://avesis.inonu.edu.tr/yayin/6762f717-2da4-430b-a02a->

b635a287b73b/determination-of-risk-factors-for-covid-19-patients-using-the-catboost-machine-learning-technique.

Yaren Aydın, Gebrail Bekdaş & Sinan Melih Nigdeli 2024, 'Effect of CatBoost Parameters on Cost Minimization of Rectangular Section Reinforced Concrete Columns Under Uniaxial Bending Effect', *Studies in Systems, Decision and Control*, Springer International Publishing, pp. 205–220.

Yingze, S, Yingxu, S, Xin, Z, Jie, Z & Degang, Y 2024, 'Comparative analysis of the TabNet algorithm and traditional machine learning algorithms for landslide susceptibility assessment in the Wanzhou Region of China', *Natural Hazards*, vol. 120, Springer Science+Business Media, no. 8, pp. 7627–7652.

Yury Gorishniy, Rubachev, I, Khrulkov, V & Babenko, A 2021, 'Revisiting Deep Learning Models for Tabular Data', *arXiv* (Cornell University), Cornell University.